

acri: A Client-Side Capability Resolver for Tool-Augmented Language Models

Piyush Sharma

Independent Researcher

github.com/INERATE/acri

Abstract—As the number of tools exposed to a language model grows past a small threshold, tool-selection accuracy degrades – Anthropic’s own documentation places the cliff at 30–50 tools [1]. Anthropic and OpenAI ship server-side tool search to address this [1], [2]; providers without native tool search (Gemini, Ollama, vLLM, and any locally-hosted model) have no equivalent. We present acri, a client-side, provider-agnostic capability resolver: given a query and a registered tool corpus, it returns the k tools relevant to that call using BM25 lexical retrieval [3], with zero training and zero external dependencies. On a 50-query, 100-tool synthetic benchmark, acri improves tool-selection accuracy from 84% (all tools shown) to 92% (top-5 shown), measured against live `gemini-2.5-flash` calls. We further show, and this is the paper’s central non-obvious result, that a naive “send fewer tokens” framing of this problem is often *wrong* once provider-side prompt caching is accounted for: cutting the tool-schema prefix at full price is more expensive than sending the full prefix at cache price unless the cut exceeds roughly tenfold. We report where our own benchmark breaks (recall@5 degrades from 100% to 92% at 500 tools) rather than hide it, and state explicitly what remains unmeasured before any version of this work could support a stronger claim. This is a technical report: it names its own evidence gaps rather than a set of results ready for peer review at a top venue.

Index Terms—tool-augmented language models, retrieval-augmented generation, prompt caching, BM25, Model Context Protocol, tool selection

I. INTRODUCTION

Tool-augmented language models select one function from a registered set on each turn. As that set grows, the model’s task – reading N schemas and picking the right one – degrades in accuracy for a well-documented reason: schema tokens compete for the same attention budget as the rest of the prompt, and confusability between similarly-named or similarly-described tools rises with N . Anthropic’s tool-use documentation names 30–50 tools as the point past which this degradation becomes visible in practice, and ships *tool search* – server-side retrieval over a large tool catalog – as the mitigation [1]. OpenAI ships an equivalent [2].

Gemini does not. Ollama does not. vLLM does not. A locally-hosted 8B model does not. Building against these gets no mitigation for free, and re-implementing server-side retrieval per provider is not a reasonable ask of an application author.

acri is that layer, implemented once, client-side: `pip install pyacri, import acri`, no gateway process, no training step, no provider lock-in. It sits between application code and whichever provider SDK is already in use, and

decides – per call – which k of the registered tools the model actually sees.

A. What this paper claims, and what it does not

Early internal drafts of this project accumulated an unsupported claim set: lower cost, faster execution, better memory, higher accuracy, no hallucination, all attributed to the same mechanism. Table I sorts every claim actually made in this work by whether it survives scrutiny, including the claims we explicitly do *not* make.

TABLE I
CLAIMS, SORTED BY DEFENSIBILITY

Claim	Status
Improves tool-selection accuracy	Measured: 84%→92%, one run, <code>gemini-2.5-flash</code> (§IV-B).
Frees context capacity	Earnable, not separately measured.
Works where no native tool search exists	A fact, not an empirical claim.
Lowers API cost	False below $\sim 10\times$ reduction against a cached baseline (§III-B).
Faster end-to-end execution	Not directly; resolver latency (p50 0.179 ms at 500 tools) is dwarfed by any network call.
Eliminates hallucinated tool calls	Not claimable; a smaller candidate set reduces likelihood, not possibility.

II. RELATED WORK

Server-side tool search. Anthropic and OpenAI solve the identical problem for providers that ship it [1], [2]. acri targets the complement – every provider and self-hosted model that does not.

MCP’s own filtering proposals. The Model Context Protocol’s enhancement process has considered this problem directly. SEP-1300 (“Tool Filtering with Groups and Tags”), SEP-1821 (“Dynamic Tool Discovery”), and SEP-1881 (“Scope-Filtered Tool Discovery”) were all closed as of late 2025 [4], [5], [6]. These are *static*, *server-declared* filters – an operator tags or groups tools ahead of time. acri performs *query-aware retrieval* – which tools are relevant changes per call, not per deployment. The mechanisms are complementary, not competing; the distinction matters for positioning, since the argument for query-aware retrieval narrows as server-declared filtering becomes more common.

Model routing. RouteLLM [7] addresses a related but distinct problem – selecting *which model* answers a query, not which tools it sees. It is the most-cited work in this space but has not been updated since August 2024. acri’s own router component (§III-C) is scoped narrowly by comparison: route only stateless, prefix-free calls, once, before generation – informed by [8], which finds a lightweight pre-generation router outperforms optimal cascade policies on 4 of 5 evaluated datasets, because a cascade has already paid the cheap model’s cost before deciding to escalate.

Adjacent implementations. `openclaw-toolsearch` [9] validates that this problem is recognized outside this work, but ships as a server-side proxy, not an importable client library, and has not taken the niche acri targets.

Serialization. TOON [10] claims 42.6% token reduction, measured against pretty-printed JSON. Against compact JSON – what every SDK actually transmits – the reduction is 14.5% on the same comparison, and TOON’s own documentation states compact JSON wins for deeply-nested, non-uniform data, which describes JSON Schema tool definitions. We do not use TOON at the schema layer for this reason; acri’s `press` component applies a TOON-style encoding at the *tool result* layer instead, where results are frequently large, uniform, and never cached.

III. SYSTEM DESIGN

A. Architecture

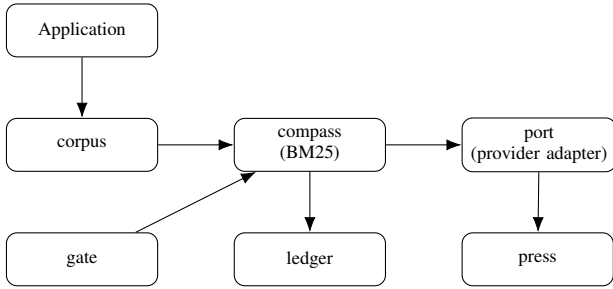


Fig. 1. acri’s components. `compass` resolves once per task; `gate` is an advisory necessity check; `press` digests large tool results; `ledger` records every resolution for the evaluation in §IV.

acri decomposes into three components sufficient to form a complete, usable system on their own (Fig. 1): **corpus** (ingests tools from MCP servers, OpenAPI specs, or plain Python callables into one searchable index, built once and reused across a task), **compass** (the resolver: BM25 lexical retrieval [3], no training, no embedding model required), and **port** (thin per-provider adapters – Gemini, and any OpenAI-compatible endpoint including self-hosted vLLM/Ollama). A **ledger** records every resolution decision (query, offered tools, model’s selection, latency, corpus size) for the accuracy measurement in §IV.

We use BM25 rather than dense embeddings deliberately: it needs no training data, no encoder model, and no vector index. Whether a dense retriever would outperform this on tool descriptions specifically is, as of this draft, an open,

unmeasured question (§V); the claim we make is that lexical retrieval is *sufficient* to produce the accuracy result in §IV-B, not that it is optimal.

B. The central architectural argument: resolve once, never rewrite

This is the paper’s primary non-obvious contribution, and the reason a naively-appealing design – re-run retrieval every conversational turn, swap the tool schema block each time – is wrong.

Every major LLM provider prices cached input tokens at roughly one-tenth of uncached input tokens, keyed on a stable prompt prefix. The tool-schema block sits at the front of the request. Rewriting it on every turn invalidates the provider’s cache on every turn – the result is that a *smaller* prompt is billed at the *full* rate, while a larger, unchanged prompt is billed at the cached rate.

Formally, let a full, uncached tool block cost C tokens at price p , and a resolved block cost rC tokens ($r < 1$) at the same uncached price, versus the full block at the cached price $p/10$ (approximating typical provider cache discounts). Re-resolving every turn is cheaper only when:

$$rCp < C \cdot \frac{p}{10} \iff r < \frac{1}{10} \quad (1)$$

That is: cutting the tool block by less than roughly tenfold, while forfeiting the cache discount to do it, is a **net regression**, not a saving. The design Eq. (1) forces: **resolve once per task, write the tool block once, append after that – never rewrite a prefix that has already been sent**. The one sanctioned exception is `find_more_tools`, a capability always offered to the model so a bad initial resolution is recoverable mid-task via a normal appended tool call, rather than by rewriting history.

Four smaller components close specific gaps, each gated on evidence of need:

- **gate** – a necessity check (does this turn need a tool at all), implemented as a threshold on the BM25 score `compass` already computes, never a second trained classifier. Advisory only: it may influence what is offered, never veto what the model actually calls, since a wrongly-suppressed tool call is silent and unlogged while a spurious offer is cheap and logged.
- **press** – large tool *results* (not schemas) become a short digest plus a recoverable handle, addressing that JSON Schema tool definitions are TOON’s documented worst case while tool results are frequently its best case.
- **sandbox** – CPU/memory/network/filesystem limits on stdio MCP servers via the host container engine. MCP already provides process isolation; the addressed gap is *resource* limits, not process separation.
- **router** – scoped to stateless, prefix-free calls only, routed once before generation, never mid-stream. Mid-stream model switching was evaluated and rejected: stream-abort billing is unmeasurable at at least one major provider, and

[11] measures a single clean handoff moving multi-turn instruction-following by double digits in *both* directions.

C. Supporting mechanisms

Four smaller components close specific gaps, each gated on evidence of need:

- **gate** – a necessity check (does this turn need a tool at all), implemented as a threshold on the BM25 score compass already computes, never a second trained classifier. Advisory only: it may influence what is offered, never veto what the model actually calls, since a wrongly-suppressed tool call is silent and unlogged while a spurious offer is cheap and logged.
- **press** – large tool *results* (not schemas) become a short digest plus a recoverable handle, addressing that JSON Schema tool definitions are TOON’s documented worst case while tool results are frequently its best case.
- **sandbox** – CPU/memory/network/filesystem limits on studio MCP servers via the host container engine. MCP already provides process isolation; the addressed gap is *resource* limits, not process separation.
- **router** – scoped to stateless, prefix-free calls only, routed once before generation, never mid-stream. Mid-stream model switching was evaluated and rejected: stream-abort billing is unmeasurable at at least one major provider, and [11] measures a single clean handoff moving multi-turn instruction-following by double digits in *both* directions.

IV. EVALUATION

All numbers in this section are produced by a script in `assay/`, this project’s convention for where a reportable number is allowed to originate.

A. Recall@k

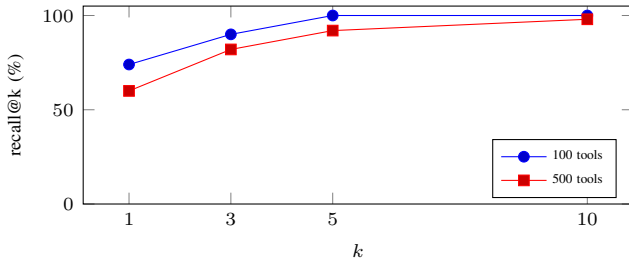


Fig. 2. Recall@k on a 100-tool corpus (52 gold queries) versus a 500-tool corpus, same queries unchanged.

Synthetic 100-tool corpus, 20 domains with deliberate cross-domain confusability, 50 hand-written queries phrased as a user would type them. Table II and Fig. 2 report recall@k both at 100 tools and after extending the corpus to 500 tools (41 new domains, same 52 gold queries, unchanged) – an honestly-reported degradation, not patched to look better. Query-by-query diagnosis attributes it to lexical competition: as the corpus grows, more tools share surface vocabulary with a given query, a limitation of BM25 specifically, not of the

resolve-once architecture in §III-B. Latency at 500 tools: p50 0.179 ms, p95 0.285 ms – three orders of magnitude below any real provider network call.

TABLE II
RECALL@K, 100 VS. 500 TOOLS

k	recall@k (100)	recall@k (500)
1	74%	60%
3	90%	82%
5	100%	92%
10	100%	98%

B. Accuracy: does the model pick correctly

Recall measures whether the correct tool is *offered*; it says nothing about whether the model *chooses* it. `assay/accuracy.py` measures this directly: naive (all 100 tools shown) versus acri (top-5 shown), same 50 queries, live `gemini-2.5-flash` calls, no mocking. This number was corrected twice in public before reaching its current value; we report the full correction history in Table III because a benchmark that always favors the tested hypothesis deserves the same suspicion as an unreceipted number.

TABLE III
ACCURACY CORRECTION HISTORY

Run	naive	acri	Cause
1	24%	53%	All tool schemas had empty properties; model correctly declined to fabricate arguments. Not a real result.
2	72%	74%	Real JSON Schema on every tool; every query supplies each declared required value.
3	84%	92%	9 queries satisfied a tool’s required fields on paper but withheld a value the action needed in practice; fixed by supplying it or promoting the field to required.

Current, final: naive 84%, acri 92%, `resolver_miss`=0 (every remaining acri-arm miss is attributable to the model’s choice, not to compass failing to offer the right tool). The naive arm’s own score moved 2 points between runs 2 and 3 with no code change on its side – we treat 2 points as this benchmark’s approximate noise floor at $n=50$, which is the central reason §V lists sample size first.

C. What this evaluation does not show

Single model (`gemini-2.5-flash`). One synthetic, LLM-generated corpus, not sourced from production MCP servers. No comparison against a dense-embedding retriever. No confidence interval or significance test on the 8-point accuracy gap.

V. LIMITATIONS

Stated as gaps to close, not hedged around:

- 1) **Sample size.** 50–52 gold queries is small for a headline claim; no bootstrap confidence interval or paired significance test has been computed. Highest-priority gap.
- 2) **Single model.** All accuracy numbers are gemini-2.5-flash only.
- 3) **No embeddings baseline.** BM25 versus dense retrieval on tool descriptions is an open question this work has not resolved.
- 4) **Synthetic corpus.** Neither corpus was sourced from a real, in-production MCP catalog.
- 5) **Re-resolution triggers are unspecified.** §III-B forbids per-turn re-resolution but does not define what should trigger `find_more_tools`, or whether that is detectable without an extra model call.

VI. CONCLUSION

We present a client-side capability resolver motivated by a documented, provider-acknowledged degradation (tool-selection accuracy past 30–50 tools) and targeted at the class of providers that ship no server-side mitigation. Its central architectural contribution is not the retrieval mechanism itself, a standard BM25 index, but the observation that provider-side prompt caching inverts the naive cost argument for doing this at all (Eq. 1). We report a measured 8-point tool-selection accuracy improvement (84%→92%, $n=50$), an honestly-reported recall degradation at $5\times$ corpus scale (100%→92% at $k=5$), and five specific, unresolved gaps – sample size foremost – that separate this technical report from a claim ready for peer review at a top venue.

REPRODUCTION

```
pip install -e ".[dev]"; python -m
assay.recall; python -m assay.scale; python
-m assay.accuracy (needs GEMINI_API_KEY);
python -m assay.diagnose. Source:
github.com/INERATE/acri. Package: pip install
pyacri. An animated walkthrough of the mechanism in
§III-B is at inerate.github.io/acri/demo.
```

REFERENCES

- [1] Anthropic, “Tool search tool,” <https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool>, 2026, states a 30–50 tool accuracy-degradation threshold. Accessed 2026-08-27.
- [2] OpenAI, “Using tools,” <https://developers.openai.com/api/docs/guides/tools>, 2026, documents `defer_loading` and `tool_search`. Accessed 2026-08-27.
- [3] S. Robertson and H. Zaragoza, “The probabilistic relevance framework: Bm25 and beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] Model Context Protocol, “Sep-1300: Tool filtering with groups and tags,” <https://github.com/modelcontextprotocol/modelcontextprotocol>, 2025, closed 2025-12-01, `state_reason: completed`.
- [5] —, “Sep-1821: Dynamic tool discovery,” <https://github.com/modelcontextprotocol/modelcontextprotocol>, 2025, closed, `state_reason: completed`.
- [6] —, “Sep-1881: Scope-filtered tool discovery,” <https://github.com/modelcontextprotocol/modelcontextprotocol>, 2025, closed, `state_reason: completed`.
- [7] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, “Routellm: Learning to route llms with preference data,” *arXiv preprint arXiv:2406.18665*, 2024.

- [8] Nguyen and Diao, “Is escalation worth it? a decision-theoretic characterization of llm cascades,” *arXiv:2605.06350*, 2026.
- [9] PyPI, “openclaw-toolsearch,” <https://pypi.org/project/openclaw-toolsearch/>, 2026, first release 2026-03-07; a server-side proxy, not a client library.
- [10] J. Schopplich, “Toon: Token-oriented object notation,” <https://github.com/toon-format/toon>, 2026, claims 42.6% fewer tokens vs. pretty JSON; compact JSON wins on nested data.
- [11] Kumar and Reyes, “Evaluating performance drift from model switching in multi-turn llm systems,” *arXiv:2603.03111*, 2026.